Introducción.-
Se ponen en práctica los conocimientos adquiridos de la librería de pandas con distintos dataframe de titanic y emisiones.

Objetivo:
1. El archivo titanic.csv contiene información sobre los pasajeros del Titanic. Escribir un programa que haga las instrucciones establecidas.
2. Los archivos emisiones-2016.csv, emisiones-2017.csv, emisiones-2018.csv y emisiones-2019.csv, contienen datos sobre las emisiones contaminantes en la ciudad de Madrid en los años 2016, 2017, 2018 y 2019 respectivamente. Escribir un programa que haga lo las instrucciones establecidas.

Herramientas.-
- Visual Studio Code

Desarrollo.-
Cabe recalcar que los comentarios también mencionen lo que hará el código.
Titanic.-

1. **Importación de la librería pandas para su uso**

```
1. import pandas as pd
```

2. **Cargar el dataset**

```
# Cargar datos
print("Generar un DataFrame con los datos del archivo.")
df = pd.read_csv("Practica_9/titanic.csv")
print(df)
```

Output:

```
Generar un DataFrame con los datos del archivo.
     PassengerId  Survived  Pclass                                               Name     Sex   Age  SibSp  Parch            Ticket     Fare Cabin Embarked
0              1         0       3                            Braund, Mr. Owen Harris    male  22.0      1      0         A/5 21171   7.2500   NaN        S
1              2         1       1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1      0          PC 17599  71.2833   C85        C
2              3         1       3                             Heikkinen, Miss. Laina  female  26.0      0      0  STON/O2. 3101282   7.9250   NaN        S
3              4         1       1       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1      0            113803  53.1000  C123        S
4              5         0       3                           Allen, Mr. William Henry    male  35.0      0      0            373450   8.0500   NaN        S
..           ...       ...     ...                                                ...     ...   ...    ...    ...               ...      ...   ...      ...
886          887         0       2                              Montvila, Rev. Juozas    male  27.0      0      0            211536  13.0000   NaN        S
887          888         1       1                       Graham, Miss. Margaret Edith  female  19.0      0      0            112053  30.0000   B42        S
888          889         0       3           Johnston, Miss. Catherine Helen "Carrie"  female   NaN      1      2        W./C. 6607  23.4500   NaN        S
889          890         1       1                               Behr, Mr. Karl Howell    male  26.0      0      0            111369  30.0000  C148        C
890          891         0       3                                 Dooley, Mr. Patrick    male  32.0      0      0            370376   7.7500   NaN        Q
```

3. Dimensiones y Estructura del Dataframe

```python
# Dimensiones y estructura del DataFrame
print("Mostrar por pantalla las dimensiones del DataFrame.")
print(df.shape)

print("Mostrar por pantalla el número de datos que contiene.")
print(df.size)

print("Mostrar por pantalla los nombres de sus columnas.")
print(df.columns)

print("Mostrar por pantalla los nombres de sus filas (índices).")
print(df.index)

print("Mostrar por pantalla los tipos de datos de las columnas.")
print(df.dtypes)

print("Mostrar por pantalla las 10 primeras filas.")
print(df.head(10))

print("Mostrar por pantalla las 10 últimas filas.")
print(df.tail(10))
```

Output:

```
Mostrar por pantalla las dimensiones del DataFrame.
(891, 12)
Mostrar por pantalla el número de datos que contiene.
10692
Mostrar por pantalla los nombres de sus columnas.
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
Mostrar por pantalla los nombres de sus filas (índices).
RangeIndex(start=0, stop=891, step=1)
Mostrar por pantalla los tipos de datos de las columnas.
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
Mostrar por pantalla las 10 primeras filas.
   PassengerId  Survived  Pclass                                                Name     Sex   Age  SibSp  Parch            Ticket     Fare Cabin Embarked
0            1         0       3                             Braund, Mr. Owen Harris    male  22.0      1      0         A/5 21171   7.2500   NaN        S
1            2         1       1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1      0          PC 17599  71.2833   C85        C
2            3         1       3                              Heikkinen, Miss. Laina  female  26.0      0      0  STON/O2. 3101282   7.9250   NaN        S
3            4         1       1        Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1      0            113803  53.1000  C123        S
4            5         0       3                            Allen, Mr. William Henry    male  35.0      0      0            373450   8.0500   NaN        S
5            6         0       3                                    Moran, Mr. James    male   NaN      0      0            330877   8.4583   NaN        Q
6            7         0       1                             McCarthy, Mr. Timothy J    male  54.0      0      0             17463  51.8625   E46        S
7            8         0       3                      Palsson, Master. Gosta Leonard    male   2.0      3      1            349909  21.0750   NaN        S
8            9         1       3  Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)  female  27.0      0      2            347742  11.1333   NaN        S
9           10         1       2                Nasser, Mrs. Nicholas (Adele Achem)  female  14.0      1      0            237736  30.0708   NaN        C
Mostrar por pantalla las 10 últimas filas.
     PassengerId  Survived  Pclass                                      Name     Sex   Age  SibSp  Parch           Ticket     Fare Cabin Embarked
881          882         0       3                          Markun, Mr. Johann    male  33.0      0      0           349257   7.8958   NaN        S
882          883         0       3              Dahlberg, Miss. Gerda Ulrika  female  22.0      0      0             7552  10.5167   NaN        S
883          884         0       2             Banfield, Mr. Frederick James    male  28.0      0      0  C.A./SOTON 34068  10.5000   NaN        S
884          885         0       3                    Sutehall, Mr. Henry Jr    male  25.0      0      0   SOTON/OQ 392076   7.0500   NaN        S
885          886         0       3       Rice, Mrs. William (Margaret Norton)  female  39.0      0      5           382652  29.1250   NaN        Q
886          887         0       2                      Montvila, Rev. Juozas    male  27.0      0      0           211536  13.0000   NaN        S
887          888         1       1               Graham, Miss. Margaret Edith  female  19.0      0      0           112053  30.0000   B42        S
888          889         0       3   Johnston, Miss. Catherine Helen "Carrie"  female   NaN      1      2       W./C. 6607  23.4500   NaN        S
889          890         1       1                       Behr, Mr. Karl Howell    male  26.0      0      0           111369  30.0000  C148        C
890          891         0       3                         Dooley, Mr. Patrick    male  32.0      0      0           370376   7.7500   NaN        Q
```

4. Exploración básica del conjunto de datos de pasajeros

```
# Pasajero con ID 148
print("Mostrar por pantalla los datos del pasajero con identificador 148.")
print(df.loc[148])

# Filas pares
print("Mostrar por pantalla las filas pares del DataFrame.")
print(df.iloc[::2])

# Nombres de personas en primera clase ordenadas alfabéticamente
print("Mostrar por pantalla los nombres de las personas que iban en primera clase ordenadas alfabéticamente.")
print(df[df['Pclass'] == 1]['Name'].sort_values())

# Porcentaje de sobrevivientes y fallecidos
print("Mostrar por pantalla el porcentaje de personas que sobrevivieron y murieron.")
print(df['Survived'].value_counts(normalize=True) * 100)

# Porcentaje de sobrevivientes por clase
print("Mostrar por pantalla el porcentaje de personas que sobrevivieron en cada clase.")
print(df.groupby('Pclass')['Survived'].mean() * 100)
```

Output:

```
Mostrar por pantalla los datos del pasajero con identificador 148.
PassengerId                                149
Survived                                     0
Pclass                                       2
Name           Navratil, Mr. Michel ("Louis M Hoffman")
Sex                                       male
Age                                       36.5
SibSp                                        0
Parch                                        2
Ticket                                  230080
Fare                                      26.0
Cabin                                       F2
Embarked                                     S
Name: 148, dtype: object
Mostrar por pantalla las filas pares del DataFrame.
     PassengerId  Survived  Pclass                                               Name     Sex   Age  SibSp  Parch            Ticket     Fare Cabin Embarked
0              1         0       3                            Braund, Mr. Owen Harris    male  22.0      1      0         A/5 21171   7.2500   NaN        S
2              3         1       3                             Heikkinen, Miss. Laina  female  26.0      0      0  STON/O2. 3101282   7.9250   NaN        S
4              5         0       3                           Allen, Mr. William Henry    male  35.0      0      0            373450   8.0500   NaN        S
6              7         0       1                            McCarthy, Mr. Timothy J    male  54.0      0      0             17463  51.8625   E46        S
8              9         1       3  Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)  female  27.0      0      2            347742  11.1333   NaN        S
..           ...       ...     ...                                               ...     ...   ...    ...    ...               ...      ...   ...      ...
882          883         0       3                      Dahlberg, Miss. Gerda Ulrika  female  22.0      0      0              7552  10.5167   NaN        S
884          885         0       3                             Sutehall, Mr. Henry Jr    male  25.0      0      0  SOTON/OQ 392076   7.0500   NaN        S
886          887         0       2                               Montvila, Rev. Juozas    male  27.0      0      0            211536  13.0000   NaN        S
888          889         0       3           Johnston, Miss. Catherine Helen "Carrie"  female   NaN      1      2        W./C. 6607  23.4500   NaN        S
890          891         0       3                                   Dooley, Mr. Patrick    male  32.0      0      0            370376   7.7500   NaN        Q

[446 rows x 12 columns]
Mostrar por pantalla los nombres de las personas que iban en primera clase ordenadas alfabéticamente.
730                    Allen, Miss. Elisabeth Walton
305                   Allison, Master. Hudson Trevor
297                     Allison, Miss. Helen Loraine
498    Allison, Mrs. Hudson J C (Bessie Waldo Daniels)
460                           Anderson, Mr. Harry
                              ...
155                    Williams, Mr. Charles Duane
351           Williams-Lambert, Mr. Fletcher Fellows
55                              Woolner, Mr. Hugh
555                            Wright, Mr. George
325                        Young, Miss. Marie Grice
Name: Name, Length: 216, dtype: object
Mostrar por pantalla el porcentaje de personas que sobrevivieron y murieron.
Survived
0    61.616162
1    38.383838
Name: proportion, dtype: float64
Mostrar por pantalla el porcentaje de personas que sobrevivieron en cada clase.
Pclass
1    62.962963
2    47.282609
3    24.236253
Name: Survived, dtype: float64
```

5. Limpieza, análisis de edad y supervivencia por clase

```
# Eliminar pasajeros sin edad
print("Eliminar del DataFrame los pasajeros con edad desconocida.")
df = df.dropna(subset=['Age'])
print(df)

# Edad media de mujeres por clase
print("Mostrar por pantalla la edad media de las mujeres que viajaban
en cada clase.")
print(df[df['Sex'] == 'female'].groupby('Pclass')['Age'].mean())

# Nueva columna: ¿menor de edad?
print("Añadir una nueva columna booleana para ver si el pasajero era
menor de edad o no.")
df['Menor'] = df['Age'] < 18
print(df[['Age', 'Menor']])

# Porcentaje de menores y mayores que sobrevivieron por clase
print("Mostrar por pantalla el porcentaje de menores y mayores de edad
que sobrevivieron en cada clase.")
print(df.groupby(['Pclass', 'Menor'])['Survived'].mean() * 100)
```

Output:

```
Eliminar del DataFrame los pasajeros con edad desconocida.
     PassengerId  Survived  Pclass                                               Name     Sex   Age  SibSp  Parch            Ticket     Fare Cabin Embarked
0              1         0       3                            Braund, Mr. Owen Harris    male  22.0      1      0         A/5 21171   7.2500   NaN        S
1              2         1       1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1      0          PC 17599  71.2833   C85        C
2              3         1       3                             Heikkinen, Miss. Laina  female  26.0      0      0  STON/O2. 3101282   7.9250   NaN        S
3              4         1       1       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1      0            113803  53.1000  C123        S
4              5         0       3                           Allen, Mr. William Henry    male  35.0      0      0            373450   8.0500   NaN        S
..           ...       ...     ...                                                ...     ...   ...    ...    ...               ...      ...   ...      ...
1              2         1       1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1      0          PC 17599  71.2833   C85        C
2              3         1       3                             Heikkinen, Miss. Laina  female  26.0      0      0  STON/O2. 3101282   7.9250   NaN        S
3              4         1       1       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1      0            113803  53.1000  C123        S
4              5         0       3                           Allen, Mr. William Henry    male  35.0      0      0            373450   8.0500   NaN        S
..           ...       ...     ...                                                ...     ...   ...    ...    ...               ...      ...   ...      ...
885          886         0       3                    Rice, Mrs. William (Margaret Norton)  female  39.0      0      5            382652  29.1250   NaN        Q
3              4         1       1       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1      0            113803  53.1000  C123        S
4              5         0       3                           Allen, Mr. William Henry    male  35.0      0      0            373450   8.0500   NaN        S
..           ...       ...     ...                                                ...     ...   ...    ...    ...               ...      ...   ...      ...
885          886         0       3                    Rice, Mrs. William (Margaret Norton)  female  39.0      0      5            382652  29.1250   NaN        Q
4              5         0       3                           Allen, Mr. William Henry    male  35.0      0      0            373450   8.0500   NaN        S
..           ...       ...     ...                                                ...     ...   ...    ...    ...               ...      ...   ...      ...
885          886         0       3                    Rice, Mrs. William (Margaret Norton)  female  39.0      0      5            382652  29.1250   NaN        Q
..           ...       ...     ...                                                ...     ...   ...    ...    ...               ...      ...   ...      ...
885          886         0       3                    Rice, Mrs. William (Margaret Norton)  female  39.0      0      5            382652  29.1250   NaN        Q
885          886         0       3                    Rice, Mrs. William (Margaret Norton)  female  39.0      0      5            382652  29.1250   NaN        Q
886          887         0       2                                 Montvila, Rev. Juozas    male  27.0      0      0            211536  13.0000   NaN        S
887          888         1       1                          Graham, Miss. Margaret Edith  female  19.0      0      0            112053  30.0000   B42        S
887          888         1       1                          Graham, Miss. Margaret Edith  female  19.0      0      0            112053  30.0000   B42        S
889          890         1       1                                  Behr, Mr. Karl Howell    male  26.0      0      0            111369  30.0000  C148        C
890          891         0       3                                    Dooley, Mr. Patrick    male  32.0      0      0            370376   7.7500   NaN        Q
```

```
Mostrar por pantalla la edad media de las mujeres que viajaban en cada clase.
Pclass
Pclass
1    34.611765
2    28.722973
3    21.750000
Name: Age, dtype: float64
Name: Age, dtype: float64
Añadir una nueva columna booleana para ver si el pasajero era menor de edad o no.
      Age  Menor
Añadir una nueva columna booleana para ver si el pasajero era menor de edad o no.
      Age  Menor
      Age  Menor
0    22.0  False
1    38.0  False
2    26.0  False
3    35.0  False
1    38.0  False
2    26.0  False
3    35.0  False
4    35.0  False
1    38.0  False
2    26.0  False
3    35.0  False
4    35.0  False
..    ...    ...
885  39.0  False
886  27.0  False
887  19.0  False
889  26.0  False
890  32.0  False

[714 rows x 2 columns]
Mostrar por pantalla el porcentaje de menores y mayores de edad que sobrevivieron en cada clase.
Pclass  Menor
1       False    63.793103
        True     91.666667
2       False    41.333333
        True     91.304348
3       False    20.216606
        True     37.179487
Name: Survived, dtype: float64
```

Emisiones 2016-2019.-

1. **Importación de la librería pandas para su uso**

```
import pandas as pd
```

2. **Generación de Dataframe y Filtración**

```
# Leer archivos CSV
print("Generar un DataFrame con los datos de los cuatro archivos.")
archivos = ['Practica_9/emisiones-2016.csv',
'Practica_9/emisiones-2017.csv', 'Practica_9/emisiones-2018.csv',
'Practica_9/emisiones-2019.csv']
df = pd.concat([pd.read_csv(archivo, sep=';') for archivo in archivos],
ignore_index=True)
print(df)

# Filtrar columnas relevantes
print("Filtrar las columnas del DataFrame para quedarse con las
columnas ESTACION, MAGNITUD, AÑO, MES y las correspondientes a los días
D01, D02, etc.")
columnas_dias = [col for col in df.columns if col.startswith('D')]
columnas = ['ESTACION', 'MAGNITUD', 'ANO', 'MES'] + columnas_dias
df = df[columnas]
print(df)
```

Output:

```
Generar un DataFrame con los datos de los cuatro archivos.
      PROVINCIA  MUNICIPIO  ESTACION  MAGNITUD PUNTO_MUESTREO   ANO  MES  D01 V01   D02 V02   D03  ...   D26 V26   D27 V27   D28 V28   D29 V29   D30 V30   D3
1  V31
0            28         79         4         1  28079004_1_38  2016    1  8.0   V   7.0   V   6.0  ...  12.0   V   9.0   V   7.0   V   8.0   V   9.0   V   9.
0    V
1            28         79         4         1  28079004_1_38  2016    2 12.0   V  13.0   V   9.0  ...   7.0   V   7.0   V   8.0   V   9.0   V   0.0   N   0.
0    N
2            28         79         4         1  28079004_1_38  2016    3 11.0   V  10.0   V   9.0  ...   9.0   V   8.0   V   7.0   V   8.0   V  10.0   V   8.
0    V
3            28         79         4         1  28079004_1_38  2016    4  8.0   V   9.0   V   9.0  ...   9.0   V   9.0   V   8.0   V   8.0   V   8.0   V   0.
0    N
4            28         79         4         1  28079004_1_38  2016    5  7.0   V   8.0   V   9.0  ...   7.0   V   7.0   V   7.0   V   7.0   V   7.0   V   7.
0    V
...         ...        ...       ...       ...            ...   ...  ...  ...  ..  ...  ..  ...  ...  ...
```

3. **Reestructura del DataFrame, Columna Fecha, eliminar fechas no válidas.**

```
# Reestructurar el DataFrame
print("Reestructurar el DataFrame para que los valores de los
contaminantes de las columnas de los días aparezcan en una única
columna.")
df = df.melt(id_vars=['ESTACION', 'MAGNITUD', 'ANO', 'MES'],
var_name='DIA', value_name='VALOR')
df['DIA'] = df['DIA'].str.extract('D(\d+)').astype(int)
print(df)

# Añadir una columna de Fecha
print("Añadir una columna con la fecha a partir de la concatenación del
año, el mes y el día.")
df['FECHA'] = pd.to_datetime(
    df[['ANO', 'MES', 'DIA']].rename(columns={'ANO': 'year', 'MES':
'month', 'DIA': 'day'}),
    errors='coerce'
)
print(df[['ANO', 'MES', 'DIA', 'FECHA']])

# Eliminar fechas no válidas y ordenar
print("Eliminar las filas con fechas no válidas (utilizar la función
isnat del módulo numpy) y ordenar el DataFrame por estaciones
contaminantes y fecha.")
df = df[~pd.isna(df['FECHA'])]
df = df.sort_values(by=['ESTACION', 'MAGNITUD', 'FECHA'])
print(df)
```

Output:

```
Reestructurar el DataFrame para que los valores de los contaminantes de las columnas de los días aparezcan en una única columna.
        ESTACION  MAGNITUD   ANO  MES  DIA  VALOR
0              4         1  2016    1    1    8.0
1              4         1  2016    2    1   12.0
2              4         1  2016    3    1   11.0
3              4         1  2016    4    1    8.0
4              4         1  2016    5    1    7.0
...          ...       ...   ...  ...  ...    ...
225241        60        14  2019    8   31   98.0
225242        60        14  2019    9   31    0.0
225243        60        14  2019   10   31   47.0
225244        60        14  2019   11   31    0.0
225245        60        14  2019   12   31    4.0

[225246 rows x 6 columns]
Añadir una columna con la fecha a partir de la concatenación del año, el mes y el día.
         ANO  MES  DIA      FECHA
0       2016    1    1 2016-01-01
1       2016    2    1 2016-02-01
2       2016    3    1 2016-03-01
3       2016    4    1 2016-04-01
4       2016    5    1 2016-05-01
...      ...  ...  ...        ...
225241  2019    8   31 2019-08-31
225242  2019    9   31        NaT
225243  2019   10   31 2019-10-31
225244  2019   11   31        NaT
225245  2019   12   31 2019-12-31

[225246 rows x 4 columns]
Eliminar las filas con fechas no válidas (utilizar la función isnat del módulo numpy) y ordenar el DataFrame por estaciones contaminantes y fecha.
        ESTACION  MAGNITUD   ANO  MES  DIA  VALOR      FECHA
0              4         1  2016    1    1    8.0 2016-01-01
7266           4         1  2016    1    2    7.0 2016-01-02
14532          4         1  2016    1    3    6.0 2016-01-03
21798          4         1  2016    1    4    6.0 2016-01-04
29064          4         1  2016    1    5    7.0 2016-01-05
...          ...       ...   ...  ...  ...    ...        ...
196181        60        14  2019   12   27   17.0 2019-12-27
203447        60        14  2019   12   28   13.0 2019-12-28
210713        60        14  2019   12   29   14.0 2019-12-29
217979        60        14  2019   12   30    5.0 2019-12-30
225245        60        14  2019   12   31    4.0 2019-12-31
```

4. **Exploración general de estaciones, contaminantes y análisis estadístico**

```python
# Estaciones y contaminantes disponibles
print("Mostrar por pantalla las estaciones y los contaminantes disponibles en el DataFrame.")
print("Estaciones:", df['ESTACION'].unique())
print("Contaminantes:", df['MAGNITUD'].unique())

# Función: emisiones por estación, contaminante y fechas
print("Crear una función que reciba una estación, un contaminante y un rango de fechas y devuelva una serie con las emisiones del contaminante dado en la estación y rango de fechas dado.")
def emisiones_estacion_contaminante_fecha(estacion, contaminante, fecha_inicio, fecha_fin):
    filtro = (
        (df['ESTACION'] == estacion) &
        (df['MAGNITUD'] == contaminante) &
        (df['FECHA'] >= fecha_inicio) &
        (df['FECHA'] <= fecha_fin)
    )

    datos_filtrados = df.loc[filtro, ['FECHA', 'VALOR']].dropna()
```

```
    if datos_filtrados.empty:
        print("No se encontraron datos para los parámetros
especificados.")
        return pd.Series(dtype='float64')

    return datos_filtrados.set_index('FECHA')['VALOR']

print(emisiones_estacion_contaminante_fecha(4, 1, '2018-01-01',
'2018-01-10'))

# Resumen descriptivo por contaminante
print("Mostrar un resumen descriptivo (mínimo, máximo, media, etc.)
para cada contaminante.")
print(df.groupby('MAGNITUD')['VALOR'].describe())
```

Output:

```
Mostrar por pantalla las estaciones y los contaminantes disponibles en el DataFrame.
Estaciones: [ 4  8 11 16 17 18 24 27 35 36 38 39 40 47 48 49 50 54 55 56 57 58 59 60]
Contaminantes: [ 1  6  7  8 12  9 10 14 20 30 35 42 43 44]
Crear una función que reciba una estación, un contaminante y un rango de fechas y devuelva una serie con las emisiones del contaminante dado en la estación y rango de
 fechas dado.
FECHA
2018-01-01    1.0
2018-01-02    1.0
2018-01-03    2.0
2018-01-04    1.0
2018-01-05    1.0
2018-01-06    1.0
2018-01-07    1.0
2018-01-08    2.0
2018-01-09    3.0
2018-01-10    1.0
Name: VALOR, dtype: float64
Mostrar un resumen descriptivo (mínimo, máximo, media, etc.) para cada contaminante.
           count       mean         std   min    25%    50%    75%       max
MAGNITUD
1        14610.0   7.428953    7.012504  0.00   4.00   7.00  10.00    610.00
6        14610.0   0.350233    0.215935  0.00   0.20   0.30   0.40     14.90
7        35064.0  20.446412  135.123509  0.00   4.00   9.00  23.00  24742.00
8        35064.0  37.677618   20.118050  0.00  22.00  35.00  50.00    148.00
9         8948.0  10.087729   10.643591  0.00   6.00   9.00  13.00    850.00
10       17897.0  18.772923   35.723619  0.00  10.00  16.00  24.00   4481.00
12       35064.0  67.959417   61.443940  0.00  29.00  48.00  84.00   1005.00
14       20454.0  49.941772   24.753120  0.00  31.00  52.00  69.00    336.00
20        8766.0   2.364944    4.236706  0.00   0.80   1.60   2.80    195.00
30        8766.0   0.531371    0.538180  0.00   0.20   0.40   0.70     15.10
35        8766.0   0.479751    1.183618  0.00   0.10   0.20   0.50     35.70
42        4383.0   1.400897    0.251836 -0.01   1.25   1.38   1.54      3.09
43        4383.0   1.292923    0.230898 -0.14   1.17   1.28   1.43      2.77
44        4383.0   0.108941    0.068776  0.00   0.06   0.10   0.14      1.31
```

5. **Funciones de análisis estadístico por estación, contaminante y tiempo**

```
# Función: resumen descriptivo por estación y contaminante
print("Crear una función que reciba una estación y un contaminante y
devuelva un resumen descriptivo de las emisiones del contaminante
indicado en la estación indicada.")
def resumen_estacion_contaminante(estacion, contaminante):
    filtro = (df['ESTACION'] == estacion) & (df['MAGNITUD'] ==
contaminante)
    return df.loc[filtro, 'VALOR'].describe()

print(resumen_estacion_contaminante(4, 1))

# Función: Emisiones medias mensuales por contaminante y año
print("Crear una función que devuelva las emisiones medias mensuales de
un contaminante y un año dados para todas las estaciones.")
def emisiones_mensuales(contaminante, año):
    filtro = (df['MAGNITUD'] == contaminante) & (df['ANO'] == año)
    return df.loc[filtro].groupby(['ESTACION',
'MES'])['VALOR'].mean().unstack()

print(emisiones_mensuales(1, 2019))

# Funcion: Medias mensuales de los distintos tipos de contaminantes
print("Crear un función que reciba una estación de medición y devuelva
un DataFrame con las medias mensuales de los distintos tipos de
contaminantes.")
def medias_mensuales_estacion(estacion):
    df_filtrado = df[df['ESTACION'] == estacion].copy()
    df_filtrado['VALOR'] = pd.to_numeric(df_filtrado['VALOR'],
errors='coerce')
    df_filtrado = df_filtrado.dropna(subset=['VALOR'])
    return df_filtrado.groupby(['MAGNITUD',
'MES'])['VALOR'].mean().unstack()

print(medias_mensuales_estacion(4))
```

Output:

```
Crear una función que reciba una estación y un contaminante y devuelva un resumen descriptivo de las emisiones del contaminante indicado en la estación indicada.
count    1461.000000
mean        7.329911
std        16.379050
min         1.000000
25%         4.000000
50%         7.000000
75%         9.000000
max       610.000000
Name: VALOR, dtype: float64
Crear una función que devuelva las emisiones medias mensuales de un contaminante y un año dados para todas las estaciones.
MES             1          2          3          4          5          6          7          8          9         10         11         12
ESTACION
4         16.580645  17.107143  14.548387   2.533333   1.967742   1.866667   4.677419   7.612903   8.000000  9.580645   9.933333   7.032258
8         11.322581   9.928571   9.096774   8.700000   9.129032   9.500000   9.419355   2.645161   4.466667  5.419355   5.433333   6.516129
17         9.096774   7.714286   6.903226   7.433333   7.709677   8.800000   9.483871  10.548387  10.233333 11.000000   4.133333   5.483871
18        13.290323  12.857143  10.870968   9.566667  10.225806  15.333333  13.064516   8.096774   8.166667  7.774194   3.366667   4.903226
24         4.967742   4.964286   4.580645   4.100000   4.032258   4.500000   4.709677   4.645161   4.633333  4.709677   4.333333   5.290323
35        16.129032  15.214286  12.741935  12.700000  11.677419  11.000000  11.032258  11.000000   7.933333 12.161290  11.933333  10.612903
36        13.193548  13.892857  13.709677   9.233333   6.000000   6.733333   7.419355   8.290323   5.533333  5.580645   6.133333   7.322581
38        13.516129  12.928571  10.483871  10.166667   7.935484   8.233333   6.903226   6.870968   8.600000  8.870968  11.033333  14.935484
40        10.838710   9.714286   9.483871   9.800000   9.290323   9.266667   9.290323   8.580645   8.233333 10.096774   7.533333   8.064516
57        13.645161  12.678571  11.064516   9.833333   8.870968   9.266667   8.032258   5.451613   5.600000  9.548387   6.433333   5.548387
Crear un función que reciba una estación de medición y devuelva un DataFrame con las medias mensuales de los distintos tipos de contaminantes.
MES             1          2          3          4          5          6          7          8          9         10         11          12
MAGNITUD
1          8.943548   8.663717   7.629032   4.350000   4.016129   4.075000   5.629032   7.104839   7.891667  13.379032   7.883333   8.346774
6          0.539516   0.471681   0.389516   0.379167   0.335484   0.311667   0.295968   0.306452   0.384167   0.472581   0.480000   0.574194
7         61.540323  42.893805  22.911290  16.616667  12.975806  13.233333  11.435484   9.645161  24.683333  47.233871  47.966667  72.298387
8         52.032258  49.495575  40.411290  37.891667  36.774194  39.625000  39.685484  37.145161  49.666667  51.314516  46.216667  52.838710
12       146.354839 115.247788  75.540323  63.291667  56.556452  59.841667  57.282258  52.008065  87.491667 123.717742 119.700000 163.620968
PS C:\Users\CAPS2\OneDrive\Escritorio\Github\Programs_Data_Science>
```

Conclusiones:

✅ Conclusión – Análisis de los datos del Titanic
Logre identificar patrones significativos relacionados con la supervivencia de los pasajeros. Se observó que la clase social y el género fueron factores clave:
Los pasajeros de primera clase y las mujeres tuvieron tasas de supervivencia notablemente superiores.

Además, la edad también influyó, con los menores de edad presentando mayor probabilidad de sobrevivir en algunas clases. Estos resultados reflejan decisiones sociales y estructurales tomadas durante la evacuación del barco, y muestran cómo los datos pueden revelar sesgos humanos en situaciones críticas.

✅ Conclusión – Análisis de las emisiones contaminantes en Madrid

El análisis de las emisiones contaminantes en Madrid entre 2016 y 2019 permitió identificar tendencias temporales y espaciales en los niveles de contaminación.

Se detectaron diferencias notables entre estaciones de medición y tipos de contaminantes, así como patrones estacionales en las emisiones mensuales.

Estos análisis son fundamentales para evaluar el impacto de las políticas medioambientales y para orientar acciones futuras que mejoren la calidad del aire en zonas urbanas. La limpieza de datos y la correcta estructuración temporal (con fechas válidas) fueron clave para poder realizar análisis precisos y útiles.